

The artifact I have chosen is the Travlr Getaways MEAN tech stack application I made for CS-465, full stack development. This was the foundation of the course, creating different components bit-by-bit for each week in the course. Travlr Getaways is supposed to be a modernized online travel booking service built on the MEAN stack (MongoDB ExpressJS, Angular, and Node.js) made to be compartmentalized and scalable. The app features a client-facing ExpressJS browser interface, a custom RESTful API, and an Angular single-page admin application. I selected this artifact because I believe it encapsulates the intermediate understanding of building larger projects with multiple-components using common technologies in modern web development. Specific components of Travlr Getaways showcase a deep level of understanding fundamental web-application design. The RESTFUL API shows proficiency in creating modern information transferring systems used regularly in online development. Additionally, the difference in how the ExpressJS client-facing app and the Angular admin application are examples of both static server-side rendering and dynamically updated content.

When referring to the course outcomes, I believe this enhancement covers all three of the expected bullet-points. Firstly, the routing on the RESTful API and the ExpressJS server are examples of developing computing solutions to solve the communication between the client and the server as well as the server and the admin page. The MEAN tech stack along with other technologies such as Mongoose show an ability to use well-known, valuable technology. Lastly, the recent changes made in the enhancement have furthered my progress towards developing a strong security mindset. The goal during the enhancement was to fix the access bugs in the user/login features as well as the server-crashing requests. Additionally, the jwt-package was implemented as is instead of manually.

One thing I greatly value was the work, frustration, and learning I experienced creating separate classes for error handling, routing, and auth. The async handler was the most difficult part to figure out and make sure none of the controllers can crash from a rejected promise. Changing over to use the express-jwt package instead of a hand-rolled token validation was a much easier change. Although it is not always a great metric, reducing the code from 25 lines to 3 along is greatly efficient and eliminates any possible errors from manual token validation. The error handling + error middleware centralized all the error handling throughout the code into one place which can be traced through the project.